

RAG Chatbot MVP - System Architecture & Launch Plan

Prepared for: Mahmoud Zughbor (mzughbor) — BYOK RAG widget MVP
Date: 2025-11-11

Executive Summary

This document is a concise, market-ready MVP specification for a BYOK (Bring Your Own Key) RAG chatbot product. Goal: let small & medium businesses add a 1-line JS widget to their website, connect their own OpenAI/Anthropic/Gemini API key, upload business documents, and get a brand-tuned AI assistant. You charge a low subscription (e.g. \$7/mo) for the platform and UX; users pay LLM tokens to providers directly. This PDF contains architecture, endpoints, UI screens list, widget snippet, Supabase/pgvector plan, Stripe integration, security checklist, concurrency design, deployment suggestions, and minimal cost/operations notes for launching fast.

Recommended Tech Stack

- Frontend: Next.js (React) — recommended over a pure MERN stack because Next.js gives SSR, API routes, and fast DX. - Widget: Tiny JS file served from CDN (Vercel / Cloudflare) that opens an iframe to chat UI or builds a lightweight UI inline. - Backend: Next.js API routes or a Node.js service (Express/Fastify) for ingestion, webhook, and chat relay logic. - Auth & DB: Supabase (Postgres) — use Supabase Auth, Storage, and Postgres with pgvector extension for vector storage. - Vector DB: pgvector (in Supabase) or Chroma if preferred; Supabase+pgvector keeps infra unified and simple for MVP. - Embeddings: Use the user's own API key (BYOK) to call embedding endpoints; cache embeddings in pgvector. - Billing: Stripe (subscriptions + one-time onboarding fees). - Hosting: Vercel for Next.js frontend & widget, Render/Fly/Heroku for backend if needed; Supabase for DB. - Optional inference: Offer managed generation (premium) using your provider account.

System Architecture (Overview)

High-level flow: 1) User signs up and pastes their LLM API key (or uses OAuth if provider supports). 2) User uploads documents (PDF, DOCX, CSV, FAQ, Zendesk export). Files stored in Supabase Storage. 3) Server-side ingestion job reads files, performs OCR if needed, chunks text, calls embeddings using user's key, and stores vectors in pgvector. 4) The user grabs their unique JS snippet from dashboard and pastes it on their site. 5) Website visitors open the widget; messages are forwarded to your API (or proxied) which retrieves top-k vectors and calls the user's LLM key to generate answers. 6) Responses returned to widget; optionally store conversation + analytics in Supabase. Concurrency: API + DB are stateless where possible; use connection pooling, job queues (BullMQ / Redis) for ingestion and heavy tasks; websockets (Supabase Realtime or Next.js + WebSocket) for live chat.

ASCII Architecture Diagram

```
[Client Website] --(widget.js snippet)--> [CDN (widget.js)] |
V (iframe / postMessage) [Chat UI @ YourDomain] <--> [Your API / Next.js]
|
|                                     |
|                                     +--> [Supabase Postgres + pgvector]
|                                     +--> [Supabase Storage (files)]
|                                     +--> [Stripe Billing]
|                                     +--> [Worker Queue (Redis/Bull)]
|                                     |
+--> (calls) [OpenAI / Anthropic] using user's API key (BYOK)
```

Key API Endpoints (MVP)

	Method	Description
	POST	Create user account (Supabase Auth integration)
	POST	Store user's LLM API key (encrypt) & validate (test call)
	POST	Upload document; store file in Supabase Storage; enqueue ingestion
	POST	Worker endpoint: chunking, OCR, embeddings (BYOK), store vectors
	GET	Public config for widget (colors, welcome, bot name)
	POST	Chat relay: retrieve vectors, build prompt, call LLM using user's key
	GET	Usage metrics, top queries, correction suggestions

Widget Snippet (to give users)

Copy & paste this into your site: Minimal inlined example (concept):

```
(function() { var s = document.createElement('script'); s.src = "https://cdn.yourdomain.com/widget.js"; s.async = true; s.setAttribute('data-bot-id', 'BOT_ID_HERE'); document.head.appendChild(s); })();
```

Minimal widget.js (concept) - opens iframe to hosted chat UI

```
(function(){ const script = document.currentScript || (function(){const s=document.scripts[document.scripts.length-1]; return s;})(); const botId = script.getAttribute('data-bot-id') || 'demo'; function openChat(){ if(document.getElementById('rm-chat-frame')) return; const frame = document.createElement('iframe'); frame.id = 'rm-chat-frame'; frame.src = 'https://app.yourdomain.com/chat?bot=' + encodeURIComponent(botId); frame.style.position = 'fixed'; frame.style.bottom = '90px'; frame.style.right = '20px'; frame.style.width = '360px'; frame.style.height = '540px'; frame.style.border = 'none'; frame.style['boxShadow'] = '0 10px 30px rgba(0,0,0,0.2)'; document.body.appendChild(frame); } // bubble const bubble = document.createElement('div'); bubble.innerText = '■ Help'; bubble.style.position='fixed'; bubble.style.bottom='20px'; bubble.style.right='20px'; bubble.style.padding='12px'; bubble.style.borderRadius='24px'; bubble.style.background='#1f7ef4'; bubble.style.color='white'; bubble.style.cursor='pointer'; bubble.onclick = openChat; document.body.appendChild(bubble); })();
```

Supabase + pgvector plan (Vector storage)

- Use Supabase Postgres with the pgvector extension to store embeddings: - Table: vectors (id, bot_id, doc_id, chunk_text, embedding vector, metadata JSON, created_at) - Store raw files in Supabase Storage (bucket per user).
- Advantages: single platform for Auth, Storage, DB, and Realtime; lower ops for MVP.
- Ingestion jobs: run in background worker (Redis + BullMQ or Supabase Edge Functions) to create embeddings using user's API key.
- Retrieval: SQL nearest-neighbor search (pgvector) with LIMIT k; optionally use a lightweight BM25 re-ranker.

Concurrency & Scaling (design for dozens of concurrent chats)

- Websocket or HTTP + polling: use iframe + postMessage + Websocket for live typing UX.
- Stateless chat API: store conversation history in Postgres; avoid per-connection state on app servers.
- Scale workers horizontally: Redis-backed queue for ingestion and batch embedding tasks.
- Use connection pooling for Postgres (pgbouncer) to handle many connections.
- CDN for widget.js (Vercel/Cloudflare) for global low-latency.
- For high concurrency, offload LLM calls directly from the browser to provider via user's key for trivial chats (optional), or proxy via your API for prompt-assembly and safety checks.

Security & Data Privacy Checklist

- Encrypt user API keys at rest (use KMS) and never log full keys.
- Provide instructions for users to create limited-scope API keys if provider supports.
- Use HTTPS everywhere; secure cookies; CSP for iframe.
- Implement rate limits and abuse detection to protect provider keys.
- Provide data retention controls and per-bot delete/export options.
- Display clear billing responsibility language: users pay provider token costs.

MVP Screens (numbered sheets for UI designer)

- 01 - Landing page / marketing / pricing
- 02 - Signup / Login (Supabase Auth)
- 03 - Dashboard (list of bots, usage, quick actions)
- 04 - Bot Create / Onboard wizard (name, colors, welcome message)
- 05 - API Key input screen (enter/test key, instructions & security)
- 06 - Document Upload / Connectors (Google Drive, Notion, file upload)
- 07 - Ingestion Jobs list (status, re-run, errors)
- 08 - Chat UI (hosted chat page that iframe opens) — public for site visitors
- 09 - Embedded widget preview + snippet generator
- 10 - Billing / Stripe subscriptions management
- 11 - Analytics (top queries, fallback answers, corrections)
- 12 - Settings (team members, bot deletion, data export)

UI sheets instructions for designer (separate files)

Create separate artboards/pages for: - Landing (desktop + mobile) - Dashboard (with bots list, search, create) - Onboarding wizard (3 steps) - Widget preview modal (with color/theme controls) - Chat UI (desktop iframe + mobile full-screen) - Billing & Stripe flow screens - Upload & Connectors screens (drag & drop) Provide components: Button primary/secondary, Input fields, Empty states, Error states, Toast messages. Keep the design simple, 2-tone palette, brand font, and 2 logo variations (square + wordmark).

MERN vs Next.js — Advice for Mahmoud

- MERN (MongoDB + Express + React + Node) is familiar and fine, but Next.js gives a faster developer experience for full-stack React: built-in API routes, SSR/SSG, edge functions, and simple deployment to Vercel. - For your goals (widget + dashboard + rapid iteration), Next.js is recommended. You still use Node libraries on the server-side. - Learning Next.js will let you touch frontend and backend in one codebase and ship faster than separate MERN setup. - If you prefer strictly separated API & frontend later, you can split to Express + React, but start with Next.js for MVP speed.

Stripe Integration (MVP)

- Use Stripe Checkout for subscription setup (simplicity) or Stripe Elements for embedded billing UI. - Plans: Free trial (14 days) -> Basic (\$7/mo BYOK cap) -> Pro (\$25/mo) -> Enterprise (custom). - Store Stripe customer id and subscription metadata in Supabase. - Webhook endpoint: /api/webhooks/stripe -> verify signature -> update subscription state in DB.

MVP Launch Checklist & Rough Timeline (2-4 weeks aggressive)

Week 0-1: - Repo setup, Next.js skeleton, Supabase project, Stripe test keys. - Minimal auth + dashboard skeleton, widget.js prototype, snippet generator. Week 1-2: - File upload & storage, ingestion worker, embeddings integration (test with dummy keys), - Simple pgvector schema, nearest-neighbor retrieval. Week 2-3: - Chat relay endpoint, widget iframe chat page, basic analytics, - Stripe integration + subscription gating, bot create flow. Week 3-4: - Polish UI, onboarding tests with 3 pilot users, secure key storage (KMS), caching & rate limits, - Deploy to production (Vercel + Supabase), run smoke tests, collect feedback.

Closing & Next Steps

This MVP specification is designed to let a small team (or Cursor) implement a production-ready BYOK RAG chatbot quickly. Next immediate actions for your team: 1) Create project repo & invite developers. 2) Provision Supabase + enable pgvector. 3) Configure Stripe account and webhook URL. 4) Build widget.js + hosted chat page. 5) Run pilot with 3 customers and iterate. If you want, I can now: - generate a separate technical checklist for the developer (detailed tasks with git issues), - or create a shorter investor-friendly 1-page summary.